# Convoy: shipment tracking prototype

Convoy is built on one rule: a date an operator confirmed and a date Convoy worked out must never look the same.

## The cut I chose and why

I built two roles and only the screens that answer one question each. The customer asks "is my order on time, and when does it arrive?": a shipment list, a detail page, a notification feed. Ops asks "which shipments do I chase first?": Today, an exceptions queue, a portfolio table and a detail page with the raw feed.

Ops covers two of the four sites, matched on origin or destination, and one shipment sits outside that perimeter on purpose: its URL says "not found" and search returns nothing. Nordvik, the hero customer, receives in Norway, outside the EU customs union. That gives one lane with real import customs and a Norwegian broker posting Norwegian statuses; sea arrivals inside the EU only need a proof of Union status (T2L).

Left out, and why:

- A map. A good one costs hours, a cheap one looks generic. The leg-by-leg timeline carries the same information.
- Real operator integrations. The brief is frontend only; a scripted live feed shows how freshness would feel.
- Document upload, analytics, dark mode, a translated UI. None changes the answer to the two questions.
- Free-form LLM search. It needs a network and a key; a canned parser is honest about what is simulated.
- Automated tests. I spent the hours on the product and checked by hand: lint, build, both roles at 1440 and 360 px, keyboard, reduced motion. A real gap.

## Where and how the AI adds value

Six features, each where someone needs it, all simulated with rules.

| Feature | Where | Based on |
|---|---|---|
| Exception management | Ops Today, exceptions queue | Delay, customs hold, silence, missing documents, port incidents; one proposed action per row. |
| Arrival estimates | Shipment detail, both roles | Confirmed delays plus typical customs time, with a confidence word and the reason. |
| Consolidation | Timeline, ops detail | Seven operator vocabularies ("Gate-in", "VSL DEP", "Fortollet") mapped to one set of milestones, raw feed beside it. |
| Daily summary | Ops Today | Live counts and ranked next actions, recomputed after every action. |
| Natural-language search | Ops portfolio | "shipments to France this week running late" becomes three chips and a filtered table; unused words are marked ignored. |

| Feature | Where | Based on |
|---|---|---|
| Proactive notices | Exceptions queue, customer banner and feed | Order, route, estimate, confidence, cause. Ops edits the draft before sending. |

Outputs are branded Convoy, not "AI": the user should trust the product and see the facts it used. Every output carries a stamp that opens a short explanation of what it was built from, and a "Why" line naming those facts. The interface never says "prototype": it is written as if shown to a client in the sector, and what is simulated is explained here and in the README.

A timestamped operator event is drawn solid, planned legs are dashed, and the span an estimate moved is an amber hatch; the legend says so. One `Estimate` component is the only code allowed to render an arrival, and if nothing moved the date it says "Planned", not "Estimated". An unknown operator status shows as "Not mapped" with its raw text; a quiet feed shows the customer "Awaiting update". I would rather show a gap than guess.

The live feed shows this over time: seven scripted operator events, one every four seconds, with pause, resume and reset. Each lands with a 300 ms highlight on what changed, a toast linking to the order and the header's "Last updated" time. When Nordhaul posts its own ETA on order 12466, that is an operator fact: it is solid, labelled "Operator ETA", and the Convoy estimate it replaces moves into an "ETA history" marked superseded.

Each dialog shows a "To" line with recipient and channel: email for operators, Convoy inbox plus email for customers. Documents follow ownership: the manufacturer issues the invoice, packing list and T2L, so ops sends those with the ERP file attached (the file picker is simulated); the carrier issues the bill of lading or CMR and the broker the customs declaration, so those are requested. The customer never receives a document request, and downloads any existing document as a one-page synthetic PDF generated in the browser.

Drafts are written in the recipient's language. Rhone Industries and Rhone Brokers work in French, so messages to them are drafted in French; the dialog shows the language, its reason and a "Write in English instead" switch. Notices are capped at 600 characters.

## Technical approach

Store facts, derive judgments. React 19, Vite, react-router for real URLs, plain CSS with custom-property tokens, Barlow, lucide; no backend, UI kit or state library: nothing to fight. The store (a context and a reducer) holds operator events, the plan, documents and what ops did. Risk, estimates, summary, search and drafts are pure functions in `src/ai/`, run through selectors, so an ops action reaches the customer view with no extra wiring. Production replaces that folder.

Risk weights are plain numbers (+4 for a customs hold, +3 for 24 hours of silence, +2 per day of slip capped at +6), so the same inputs give the same ranking. The demo clock is fixed at 13 Oct 2026, 14:30, and the live feed stamps its events on that clock, never the real one. 24 shipments, seven operators, four sites, identical on any machine.

In production an LLM maps free-text statuses to milestones against a schema; rules stay as fallback and low confidence lands in "Not mapped". Arrival estimates come from a model on lane history and port feeds; thresholds decide between a date, a range or "no update yet". The model drafts notices and a human approves every message to a customer. Search returns the filter object the dropdowns use,

so chips stay inspectable. A labelled set of real statuses and past delays scores each change before release.

## Assumptions, limitations and next steps

I assumed one customer and one ops persona are enough to show scoping; an English UI, metric units and EUR; operators that expose timestamped events; and a local demo, so no deployment or auth.

Limitations:

- Reload resets all data.
- The live feed is a seven-event script, not a connection.
- Search understands a fixed set of phrases.
- Risk and estimates are rules, not models.
- Drafts exist in English and French only.
- No auth, no tests.

Next steps, in order:

1. Real events behind a thin API with webhooks.
2. An evaluation set for the status mapper, before any LLM touches customer-facing text.
3. Tests for the risk and search rules, the cheapest to cover.
4. Notification preferences, document upload and a map, once I know what customers ask for.

I used AI coding tools (Claude Code) throughout. I made the scope, product and visual decisions and reviewed what it produced.